

Day 6 Summary

Dependencies — First Steps · Classes as Dependencies

1. Dependencies — First Steps

Dependency Injection lets an endpoint declare something it needs, and FastAPI provides it automatically. Main uses: share logic/resources (DB connections) across endpoints, handle security/auth, and cut repeated code.

A dependency is usually just a function

```
async def common_parameters(
    q: str | None = None,
    skip: int = 0,
    limit: int = 100,
):
    return {
        "q": q,
        "skip": skip,
        "limit": limit,
    }

@app.get("/items/")
async def read_items(
    commons: Annotated[dict, Depends(common_parameters)]
):
    return commons
```

A dependency function can take parameters exactly like a path operation function — FastAPI parses/validates them from the request the same way (query params, defaults, types).

Resolution order

- FastAPI sees `Depends(common_parameters)` on the parameter
- It inspects `common_parameters`' own parameters (`q`, `skip`, `limit`)
- It extracts + validates those values from the incoming request
- It calls `common_parameters(...)` with those values
- The returned value is injected into the endpoint's `commons` parameter
- The endpoint function then runs normally

Pass the function, don't call it

`Depends(common_parameters)` is correct — `Depends(common_parameters())` is wrong, since that would call the function immediately instead of letting FastAPI call it when resolving.

Sub-dependencies

Dependencies can depend on other dependencies, forming a tree. FastAPI resolves the whole chain automatically (e.g. `get_current_user` depends on `get_token`; an endpoint depends on `get_current_user`). This pattern is central to auth/security later.

```
def get_token():
    return "abc123"

def get_current_user(
    token: str = Depends(get_token)
):
    return {"token": token}

@app.get("/profile")
```

```
def profile(
    user = Depends(get_current_user)
):
    return user
```

Other notes

- Dependencies (and the path operation) can be either async def or plain def — FastAPI handles both
- Dependency parameters, validations, and sub-dependencies all get included in the OpenAPI schema/docs automatically

2. Classes as Dependencies

Returning a plain dict from a dependency works, but editors can't autocomplete dict keys/values since they don't know the shape. Classes fix this.

What makes something a dependency: being "callable"

A callable is anything Python can invoke with (). Functions are callable, but so are classes — instantiating `fluffy = Cat(name="Mr Fluffy")` is literally "calling" `Cat`. So a class can be used directly as a dependency: FastAPI inspects its `__init__` parameters exactly like it would a dependency function's parameters (including sub-dependencies).

Class-based dependency example

```
class CommonQueryParams:
    def __init__(
        self,
        q: str | None = None,
        skip: int = 0,
        limit: int = 100,
    ):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items/")
async def read_items(
    commons: Annotated[
        CommonQueryParams,
        Depends(CommonQueryParams),
    ]
):
    ...
```

FastAPI calls `CommonQueryParams(...)` to create an instance, and that instance (not a dict) is passed in as `commons` — now `commons.q`, `commons.skip`, `commons.limit` get full editor autocomplete and type checking.

Type annotation vs. `Depends()` — two different jobs

In `commons: Annotated[CommonQueryParams, Depends(CommonQueryParams)]`, the class name appears twice. The one inside `Depends()` is what FastAPI actually calls and extracts parameters from — that's the real dependency declaration. The type annotation itself (the first `CommonQueryParams`) has no special meaning to FastAPI's logic — it exists purely so your editor/type-checker knows what type `commons` will be. You could even write `Annotated[Any, Depends(CommonQueryParams)]` and it would still work identically at runtime, just without editor support.

The shortcut: `Depends()`

When the dependency is specifically a class that FastAPI should instantiate, you can avoid repeating the class name twice:

```
commons: Annotated[
    CommonQueryParams,
    Depends()
]
```

FastAPI infers the class to call from the type annotation itself. This is purely a convenience to reduce repetition — the docs explicitly say to ignore it if it seems more confusing than helpful; it changes nothing about behavior.

Source: fastapi.tiangolo.com/tutorial/dependencies/ and [/tutorial/dependencies/classes-as-dependencies/](https://fastapi.tiangolo.com/tutorial/dependencies/classes-as-dependencies/)